Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Hash fonksiyonlarının uygulamaları ve istenen özellikleri (ekrandan, 00:10:00)
  - Hash fonksiyonu değişken uzunluklu ikili bir girdiyi sabit uzunluklu bir çıktıya dönüştürür: $h : \{0,1\}^* \to \{0,1\}^d$. Yıldız, girdi uzunluğunun bilinmediğini gösterir (örneğin 5, 10 ya da 128 haneli bir parola).
  - Örnek senaryo: sunucuya yüklenen bir m_1 dosyasının gerçekten o dosya olup olmadığı doğrulanmak istenir. Kötü niyetli biri $h(m_1) = h(m_2)$ olan farklı bir m_2 üretebilirse doğrulama işe yaramaz.
  - MD4, MD5, SHA-1, SHA-256 gibi algoritmalar hash fonksiyonlarından yararlanır. Hepsi kaba kuvvetle (brute force) kırılabilir; güvenlik düzeyi, çakışma bulmak için gereken deneme sayısıyla ölçülür. Hoca örnek olarak MD5'in beklenenden çok daha az denemeyle kırılabildiğinin gösterildiğini söyledi.
  - Bu dersteki uygulamalar doğrudan kriptografik değildir; amaç, hangi uygulamada hangi özelliğin önemli olduğunu tartışmaktır.
- İstenen özellikler (desirable properties) (ekrandan, 00:20:00 ve 00:30:00)
  - Tek yönlülük (one-way, OW): $y \in \{0,1\}^d$ verildiğinde $h(x) = y$ olan bir x bulmak zor olmalıdır. $x \to h(x)$ kolay, geri dönüş zor olmalıdır.
  - Örnek: $h(x) = x \bmod p$ tek yönlü değildir; y bilinirse $x = y + a \cdot p$ biçiminde a değerleri denenerek x bulunur.
  - Çakışmaya dayanıklılık (collision resistant, CR): $x \neq x'$ ve $h(x) = h(x')$ olan bir (x, x') çifti bulmak uygulanabilir (feasible) olmamalıdır.
  - Hedefli çakışmaya dayanıklılık (target collision resistant, TCR): x verildiğinde $h(x) = h(x')$ olan farklı bir x' bulmak uygulanabilir olmamalıdır. CR, TCR'yi gerektirir: CR $\Rightarrow$ TCR.
  - Sözde rastgelelik (pseudo-random): fonksiyon rastgele gibi görünmelidir. Önceki derslerde asal sayılarla fonksiyon üretilmesinin nedeni budur.
  - Biçimlendirilemezlik (non-malleability, NM): $h(x)$ verildiğinde, x ile ilişkili bir x' (örneğin $x' = x + 1$) için $h(x')$ üretilememelidir.
  - Hangi özelliğin önemli olduğu uygulamaya bağlıdır. Programlama dilindeki bir sözlük (lookup table) için hız güvenlikten önemlidir; birkaç çakışma zincirlemeyle çözülür, SHA gibi karmaşık bir fonksiyon gerekmez.
- Özellikler arasındaki ilişki: iki karşı örnek
  - Başlangıç: h, hem OW hem TCR olan bir fonksiyondur.
  - OW korunur, CR bozulur (ekrandan, 00:36:30): h'nin ilk girdi biti x_1 yerine $a \oplus b$ (XOR) verilir: $h'(a, b, x_2, \ldots, x_n) = h(a \oplus b, x_2, \ldots, x_n)$. Diğer bitler değişmediği için tek yönlülük bozulmaz; ama $h'(a = 1, b = 1, x_2, \ldots, x_n) = h'(a = 0, b = 0, x_2, \ldots, x_n)$ olduğundan bol çakışma vardır.
  - OW bozulur, TCR korunur (ekrandan, 00:36:30): $h''(x) = 0 \,\|\, x$ ($|x| \leq n$ ise), aksi hâlde $h''(x) = 1 \,\|\, h(x)$. Kısa girdilerde çıktı, girdinin başına 0 eklenmiş hâlidir; baştaki biti silen biri x'i bulur. Başı 0 olan çıktılar zaten birbirinden farklıdır; başı 1 olanlar h'nin TCR özelliğini taşır. Bu yüzden TCR bozulmaz.
- Uygulamalar (ekrandan, 00:50:00, 01:00:00 ve 01:07:20)
  - Parola saklama (password storage): bilgisayarda parolanın kendisi değil $h(PW)$ saklanır. En önemli özellik OW'dir: dosyayı ele geçiren biri hash'ten parolaya dönememelidir. CR ve TCR de gereklidir ama uzun parolalarda çakışma ihtimali düşüktür.
  - Dosya değişikliği algılayıcı (file modification detector): her dosya F için $h(F)$ güvenli

bir yerde saklanır; $h(F)$ yeniden hesaplanarak dosyanın değişip değişmediği anlaşılır. Dosya zaten açık olduğu için OW önemli değildir. Önemli olan, saldırganın $h(F) = h(F')$ olan bir F' üretememesidir (tahtada TCR yazılı).
  - Dijital imza (digital signature): açık anahtar PK ve özel anahtar SK vardır. İmzalama $\sigma = \text{sign}(SK, M)$; doğrulama $\text{verify}(M, \sigma, PK)$ doğru/yanlış döndürür. Saldırgan, $h(M) = h(M')$ olan başka bir M' bulursa imzalananın başka belge olduğunu iddia edebilir; bu yüzden TCR (çakışmaya dayanıklılık) önemlidir. Belgeler arasında x, $x + 1$ gibi matematiksel ilişki yoksa NM önemli değildir.
  - Taahhüt (commitment), örneğin kapalı zarf usulü ihale: A kişisi x teklifi için $C(x)$ hesaplar ve B'ye gönderir; süre bitince A, $C(x)$'i açarak x'i gösterir. $C(x)$ x'i açığa çıkarmamalı (OW) ve sistemi A'ya karşı da korumalıdır: A, aynı hash'i veren başka bir teklif verdiğini iddia edememelidir (CR). Teklifler sayı olduğu için NM de önemlidir: $C(x)$'ten $C(x + 1)$ üretilememelidir.
- Dinamik programlama (dynamic programming) (ekrandan, 01:20:00)
  - Böl ve fethet (divide and conquer) gibi problemi alt problemlere ayırır. Ama böl ve fethet örtüşen (overlapping) alt problemleri tekrar tekrar çözer; dinamik programlama her alt problemi bir kez çözüp sonucunu saklar. Daha çok optimizasyon problemlerinde kullanılır. "Programlama" kelimesi kod yazmak anlamında değildir.
  - Dört adım (slayttan): (1) alt problemleri tanımla, (2) alt problem çözümlerini ilişkilendir, (3) özyineleme ve memoization ya da aşağıdan yukarı (bottom-up) DP tablosu kur, (4) asıl problemi çöz.
  - Karmaşıklık: alt problem sayısı $\times$ her alt problemi çözme süresi.
- Fibonacci sayıları (ekrandan, 01:25:05)
  - $F_1 = F_2 = 1$, sonrakiler önceki ikisinin toplamı. Naif özyinelemede F_{n-2}, F_{n-3} gibi değerler tekrar tekrar hesaplanır; üst sınır $T(n) = \sum_{i=0}^{n-1} 2^i = O(2^n)$, sıkı sınır altın oran üzeri n'dir.
  - Memoized sürüm: `n` sözlükte varsa `memo[n]` döndürülür; $n \leq 2$ ise $f = 1$, değilse $f = fib(n-1) + fib(n-2)$; sonra `memo[n] = f`. n alt problem, her biri $O(1)$; toplam $O(n)$.
- En uzun palindrom alt dizisi (longest palindromic subsequence) (ekrandan, 01:30:00 ve 01:40:00)
  - Bir $x[1..n]$ dizisinde, karakterlerin ardışık olması gerekmeyen en uzun palindrom alt dizi aranır. Örnek: "character" içinde "carac" (uzunluk 5); "ara" da bir palindromdur. Her tek karakter 1 uzunluklu palindromdur.
  - $L(i, j)$: $x[i..j]$ içindeki en uzun palindrom alt dizinin uzunluğu ($i \leq j$). $i = j$ ise 1; $x[i] = x[j]$ ve $i + 1 = j$ ise 2; $x[i] = x[j]$ ise $2 + L(i + 1, j - 1)$; değilse $\max(L(i + 1, j), L(i, j - 1))$.
  - En kötü durum bütün karakterlerin farklı olmasıdır (örnek: ABCDEF, indisler 0..5). Özyineleme ağacında her seviyede düğüm sayısı ikiye katlanır (1, 2, 4, 8, ...) ve aynı (i, j) çiftleri tekrar hesaplanır; tahtadaki sonuç $\Theta(2^{n-1})$ (ekrandan, 01:38:20).
  - Memoization: özyinelemeden önce "$L[i, j]$ hesaplandıysa ona bak, özyineleme yapma"; her `return` noktasında tablo doldurulur. Alt problem sayısı $i < j$ çiftleri kadar, yani $\binom{n}{2}$; her biri $O(1)$; toplam $\Theta(n^2)$ (ekrandan, 01:42:10).
- Çubuk kesme (rod cutting) (ekrandan, 01:50:00 ve 01:55:50)
  - n uzunluklu çubuk tam sayı uzunluklu parçalara kesilip satılır; her uzunluğun fiyatı p_i tabloda verilir; kesme maliyeti yoktur; en yüksek gelir aranır. Kesim sayısı 0 ile $n - 1$ arasındadır.
  - Fiyat tablosu: uzunluk 1..8 için p_i = 1, 5, 8, 9, 10, 17, 17, 20.
  - 4 uzunluklu çubuk: 4 → 9; 1,3 → 9; 2,2 → 10; 3,1 → 9; 1,1,2 / 1,2,1 / 2,1,1 → 7; 1,1,1,1 → 4. En iyisi 2+2 (10).
  - Her birim noktada kesim ya vardır ya yoktur; 2^{n-1} seçenek vardır ($n = 4$ için 8).
  - En iyi gelirler r_i: $r_1 = 1$, $r_2 = 5$, $r_3 = 8$ (kesimsiz), $r_4 = 10$ (2+2), $r_5 = 13$ (2+3), $r_6 = 17$ (6), $r_7 = 18$ (1+6 ya da 2+2+3), $r_8 = 22$ (2+6).
  - Genel bağıntı: önce $r_k = \max(p_k, r_1 + r_{k-1}, r_2 + r_{k-2}, \ldots, r_{k-1} + r_1)$; sadeleşmiş hâli $r_k = \max_{1 \leq i \leq k}(p_i + r_{k-i})$, $r_0 = 0$ (ekrandan, 02:00:00).
- Çubuk kesmenin naif özyinelemeli çözümü (ekrandan, 02:10:00 ve 02:16:25)
  - `Cut-Rod(p, n)`: $n = 0$ ise 0 döndür; q = MinInt; $i = 1..n$ için $q = \max(q, p(i) + \text{Cut-Rod}(p, n - i))$; q döndür.

  - $CR(p,4)$; $CR(p,3)$, $CR(p,2)$, $CR(p,1)$, $CR(p,0)$'ı çağırır; bunlar da aynı alt problemleri tekrar çağırır.
  - $T(n) = 1 + \sum_{i=1}^{n} T(n-i)$, $T(0) = 0$: $T(1) = 1$, $T(2) = 2$, $T(3) = 4$, genel olarak $T(n) = 2^{n-1}$. Üstel karmaşıklık.
- Memoized ve aşağıdan yukarı çubuk kesme (ekrandan, 02:20:00, 02:27:20 ve 02:30:00)
  - `Memorized-Cut-Rod(p, n)`: $r[0..n]$ dizisi kurulur; $r(0) = 0$, diğerleri MinInt ($-\infty$); `Memorized-Cut-Rod-Aux(p, n, r)` çağrılır.
  - Aux: $r(n) =$ MinInt ise (henüz hesaplanmamışsa) $q =$ MinInt; $i = 1..n$ için $q = \max(q, p(i) + \text{Aux}(p, n-i, r))$; sonra $r(n) = q$; $r(n)$ döndürülür.
  - Aşağıdan yukarı: $r(0) = 0$; $j = 1..n$ için $q =$ MinInt; $i = 1..j$ için $q = \max(q, p(i) + r(j-i))$; $r(j) = q$.
  - İki sürüm de $\Theta(n^2)$: $\sum_{j=1}^{n} \sum_{i=1}^{j} 1 = \sum_{j=1}^{n} j = n(n+1)/2$. Memoized sürümde n alt problem, her biri en fazla n dönüş.
- Çözümün kendisini bulmak (ekrandan, 02:34:00)
  - Yukarıdaki algoritmalar yalnız en yüksek geliri verir, kesim yerlerini vermez. `Extended-Bottom-Up-Cut-Rod` ek bir $s[0..n]$ dizisinde her uzunluk için en iyi ilk kesimi tutar: `if q < p(i) + r(j-i) then q := p(i) + r(j-i); s(j) := i`.
  - `Print-Cut-Rod-Solution`: $n > 0$ olduğu sürece $s(n)$ yazdırılır ve $n := n - s(n)$ yapılır.
  - Zaman karmaşıklığı değişmez; yalnız bellek karmaşıklığı artar.

Hocanın Özellikle Vurguladığı Kısımlar

- Özelliklerin önemi uygulamaya bağlıdır
  - Parola saklamada OW, dosya değişikliği ve dijital imzada çakışmaya dayanıklılık, ihale (commitment) uygulamasında OW, CR ve NM birlikte önemlidir. Sözlük gibi uygulamalarda hız güvenlikten önce gelir.
- Özellikler birbirini gerektirmez
  - CR, TCR'yi gerektirir; ama OW ile CR/TCR arasında böyle bir bağ yoktur. XOR ve $0 \,\|\, x$ örnekleri bunu gösterir.
- Dinamik programlamanın özü tekrar eden hesaplamadan kaçınmaktır
  - Fibonacci'de üstelden doğrusala, palindrom alt dizisinde ve çubuk kesmede üstelden karesele inilir.
- Karmaşıklık formülü
  - Dinamik programlamada süre = alt problem sayısı $\times$ alt problem başına süre. Özyinelemeli (memoized) ya da aşağıdan yukarı yazmak karmaşıklığı değiştirmez.
- Çubuk kesmede değer ile çözüm farklıdır
  - En yüksek geliri bulmak kesim yerlerini vermez; bunun için ayrı bir dizi tutulmalıdır.

Kısa Tekrar Notları

- $h : \{0,1\}^* \to \{0,1\}^d$: değişken uzunluklu girdi, sabit uzunluklu çıktı.
- OW: y verilince $h(x) = y$ olan x bulmak zor. x mod p tek yönlü değildir.
- CR: $x \neq x'$, $h(x) = h(x')$ çifti bulmak zor. TCR: verilen x için böyle bir x' bulmak zor. CR $\Rightarrow$ TCR.
- NM: $h(x)$'ten ilişkili x' için $h(x')$ üretilememeli.
- $h(a \oplus b, x_2, \dots)$: OW kalır, CR bozulur. $h''(x) = 0 \| x \,/\, 1 \| h(x)$: OW bozulur, TCR kalır.
- Parola: OW. Dosya değişikliği: (T)CR. Dijital imza: (T)CR. Commitment: OW + CR + NM.
- DP adımları: alt problem, ilişki, memoization ya da bottom-up tablo, asıl problem.
- DP süresi: alt problem sayısı $\times$ alt problem başına süre.
- Fibonacci: naif $O(2^n)$, memoized $O(n)$.
- Palindrom alt dizisi: naif üstel, memoized $\Theta(n^2)$.
- Çubuk kesme: $r_k = \max_{1 \le i \le k}(p_i + r_{k-i})$, naif 2^{n-1}, memoized ve bottom-up $\Theta(n^2)$; kesimler için s dizisi.

Detaylı Açıklamalar

- Hash fonksiyonu tasarımında amaç, girdiden çıktıya geçişin hızlı, geriye dönüşün zor olmasıdır. Hoca, şimdiye kadar derste kullanılan x mod p gibi fonksiyonların tek yönlü

olmadığını gösterdi: çıktı y bilindiğinde $x = y + a \cdot p$ biçimindeki adaylar sırayla denenerek girdi bulunabilir. Tek yönlülüğü sağlamak için fonksiyona geri dönüşü zorlaştıran katmanlar eklenir; fonksiyon karmaşıklaştıkça tek yönlülük de güçlenir.

- CR ile TCR arasındaki fark, saldırganın elinde ne olduğudur. CR'de saldırgan aynı hash'i veren herhangi iki farklı girdiyi serbestçe arar; TCR'de bir girdi verilmiştir ve onunla çakışan ikinci bir girdi aranır. Herhangi bir çifti bulmak, belirli bir girdiye eş bulmaktan kolay olduğu için CR'ye sahip bir fonksiyon TCR'ye de sahiptir.
- Ders sonrası not: Tahtadaki TCR tanımında "given x, to find $x = x'$" yazıyor; doğrusu $x \neq x'$ olan ve $h(x) = h(x')$ sağlayan bir x' bulmaktır (ekrandan, 00:20:00).
- İlk karşı örnekte h''nin girdisi bir bit uzar: a ve b ayrı ayrı alınır, XOR'ları h'ye ilk bit olarak verilir. $(a, b) = (1, 1)$ ile $(0, 0)$ aynı XOR'u (0) verdiği için diğer bitler aynı kaldıkça bu iki girdi her zaman çakışır; çakışma bulmak kolaylaşır. Ancak h''nin çıktısından geri dönmek hâlâ h'yi tersine çevirmeyi gerektirdiği için OW korunur.
- İkinci karşı örnekte h'' kısa girdilerde girdiyi neredeyse açıkça yayımlar; bu yüzden OW bozulur. Kısa girdilerin çıktıları girdinin kendisini içerdiği için birbirinden farklıdır; uzun girdilerin çıktıları h'nin çıktısıdır ve başlarındaki 1 biti onları kısa girdilerin çıktılarından ayırır. Böylece TCR bozulmadan kalır.
- Parola saklama tartışmasında bir öğrenci, uzun bir parolanın da kısa bir parolayla aynı hash'i verebileceğini, yani çakışmanın parola uzunluğundan bağımsız olduğunu söyledi. Hoca, fonksiyonun tasarımına bağlı olarak bunun mümkün olduğunu kabul etti; ama yüksek entropili (örneğin 128 bitlik) bir parolanın kaba kuvvetle bulunmasının daha zor olduğunu, uzun parolanın her zaman iyi olduğunu belirtti.
-
- Dijital imzada özel anahtar ile açık anahtar matematiksel olarak ilişkili olmalı ama biri diğerini açığa çıkarmamalıdır; bu konu kriptoloji dersinin kapsamındadır. Bir öğrenci aynı imzanın başka bir açık anahtarla doğrulanması senaryosunu sordu; hoca bunun da farklı girdilerin aynı hash'i vermesi durumuna, yani yine çakışmaya dayanıklılığa dayandığını söyledi.
- İhale (commitment) örneğinde zarfın dışı ($C(x)$) herkese açıktır, içi (x) değildir. Sistem hem dışarıdan bakanlara (OW) hem teklif verenin kendisine (CR) karşı korunmalıdır: örneğin 20 milyon teklif eden biri sonradan 15 milyon yeterliymiş diye "15 demiştim" diyememelidir. Teklifler sayı olduğu için başka bir teklif verenin $C(x)$'ten $C(x + 1)$ üretip ihaleyi az farkla kazanması da engellenmelidir (NM).
- Dinamik programlamada memoization, özyinelemeli çözümde hesaplanan değerin bir sözlükte ya da dizide saklanmasıdır; aşağıdan yukarı çözümde aynı iş DP tablosuyla yapılır. Fibonacci'de n farklı alt problem vardır ve her biri sabit sürede çözülür; bu yüzden naif çözümün üstel süresi $O(n)$'e iner.
- Ders sonrası not: Hoca ve slayt "memorization" diyor; literatürdeki terim memoization'dır. Ayrıca hoca naif Fibonacci için altın oran üzeri n'i "ortalama" durum olarak andı; büyük olasılıkla kastedilen, ağaç yaprakları aynı seviyede bitmediği için çalışma süresinin 2^n üst sınırından daha sıkı olan $\Theta(\varphi^n)$ olmasıdır.
- Palindrom alt dizisi örneğinde her çağrı ya i'yi artırır ya j'yi azaltır; bu yüzden $(1, 4)$, $(2, 4)$, $(1, 3)$ gibi çiftler ağacın farklı dallarında tekrar tekrar hesaplanır. Bir lookup table ile her çift bir kez hesaplanınca alt problem sayısı $i < j$ çiftlerinin sayısıyla sınırlanır.
- Ders sonrası not: Tahtadaki örnek dizi ABCDEF altı karakterlidir (indis 0..5); tahtadaki $n = 5$ son indistir. Yazılan kodla tüm karakterleri farklı altı karakterli bir dizide toplam 63 çağrı yapılır (python ile sayıldı); sonuç yine üsteldir.
- Çubuk kesmede tablo küçükten büyüğe doldurulur. r_3 hesaplanırken p_3, $p_1 + p_2$, $p_1 + p_1 + p_1$ karşılaştırılır; $p_1 + p_2$ yerine $r_1 + r_2$ yazılabildiği fark edilince formül gelirler cinsinden genelleşir. Sonra ilk parça fiyat (p_i), geri kalan kısım gelir (r_{k-i}) olarak alınıp $r_k = \max(p_i + r_{k-i})$ sade biçimine ulaşılır. r_4 için $p_1 + r_3$, $p_2 + r_2$, $p_3 + r_1$, $p_4 + r_0$ karşılaştırılır. Tablo değerleri ve 4 uzunluklu çubuğun 8 seçeneği python ile doğrulandı.

- Naif `Cut-Rod`'un çağrı sayısı $T(n) = 1 + \sum_{i=1}^{n} T(n-i)$ yinelemesinden gelir: her çağrı bir işlem yapar ve n alt çağrı açar. Değerler 0, 1, 2, 4, 8, ... diye ikiye katlanır (python ile doğrulandı). Memoized sürüm aynı alt problemleri tekrar çözmez; her $r(n)$ bir kez doldurulur.
- `Extended-Bottom-Up-Cut-Rod` örnek tabloda $s = (1, 2, 3, 2, 2, 6, 1, 2)$ verir ($n = 1..8$); $n = 8$ için yazdırılan kesimler 2 ve 6'dır (python ile hesaplandı).
- Ders sonrası not: Tahtadaki aşağıdan yukarı kodda dış döngü "for j in i..n" yazılmış; doğrusu slayttaki gibi "for j in 1..n"dir (ekrandan, 02:27:20 ve 02:34:00).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.